

Assignment 4

Due Thursday October 4 at 11:59pm on Blackboard

As before, the questions without solutions are an assignment: you need to do these questions yourself and hand them in (instructions below). The assignment is due on the date shown above. An assignment handed in after the deadline is late, and may or may not be accepted (see course outline). My solutions to the assignment questions will be available when everyone has handed in their assignment.

You are reminded that work handed in with your name on it must be *entirely your own work*.

Assignments are to be handed in on Quercus. See <https://www.utoronto.ca/~butler/c32/quercus1.nb.html> for instructions on handing in assignments in Quercus. Markers' comments and grades will be available there as well.

As ever, begin with this:

```
library(tidyverse)
```

1. A researcher is trying to design a maze that can be run by rats in about 60 seconds. One particular maze was run by a sample of 21 rats, with the times shown in <http://www.utoronto.ca/~butler/c32/maze.txt>.
 - (a) Read the data into R. What (if anything) are the data values delimited by?
 - (b) Run a sign test, doing it yourself as we did in class: count the number of values above and below 60, take the *smaller* of those, and find the probability of a value of that or smaller still on a binomial distribution with $n = 21$ and $p = 0.5$ (we have 21 data points), doubling the answer because the test is two-sided.

Solution:

Count how many values are above and below 60:

```
times %>% count(time>60)

## # A tibble: 2 x 2
##   `time > 60`      n
##   <lgl>         <int>
## 1 FALSE          16
## 2 TRUE           5
```

5 above and 16 below. Then find out how likely it is that a binomial with $n = 21, p = 0.5$ would produce 5 or fewer successes:

```
p=sum(dbinom(0:5,21,0.5))
p
## [1] 0.01330185
```

or if you prefer count upwards from 16:

```
sum(dbinom(16:21,21,0.5))  
## [1] 0.01330185
```

and double it to get a two-sided P-value:

```
2*p  
## [1] 0.0266037
```

We'll compare this with `smmr` in a moment.

- (c) Install my package `smmr`, if you haven't already. To do this, you first need to install the package `devtools` (if you haven't already), by going to the console and typing

```
install.packages("devtools")
```

When that's all done, install `smmr` thus:

```
library(devtools)  
install_github("nxskok/smmr")
```

That all needs to be done only once. Then, each R Studio session where you want to use `smmr` needs this:

```
library(smmr)
```

As usual, only the `library` thing only needs to be done every time.

When you have `smmr` installed, use `sign_test` from that package to re-run your sign test. Do you get the same P-value?

Solution: The sign test function takes a data frame, an (unquoted) column name from that data frame of data to test the median of, and a null median (which defaults to 0 if you omit it):

```
library(smmr)  
sign_test(times,time,60)  
  
## $above_below  
## below above  
##    16     5  
##  
## $p_values  
##   alternative    p_value  
## 1         lower 0.01330185  
## 2          upper 0.99640131  
## 3    two-sided 0.02660370
```

This shows you two things: a count of the values below and above the null median, and then the P-values according to the various alternative hypotheses you might have.

In our case, we see again the 16 maze-running times below 60 seconds and 5 above (one of which was a long way above, but we don't care about that here). We were testing whether the median was different from 60, so we look at the two-sided P-value of 0.0266, which is exactly what we had before.

If `sign_test` doesn't work for you (perhaps because it needs a function `enquo` that you don't have), there is an alternative function `sign_test0` that doesn't use it. It requires as input a *column* of values (extracted from the data frame) and a null median, thus:

```
sign_test0(times$time, 60)

## $above_below
## below above
##      16      5
##
## $p_values
##   alternative    p_value
## 1         lower 0.01330185
## 2          upper 0.99640131
## 3    two-sided 0.02660370
```

The output should be, and here is, identical.

- (d) Package `smmr` also has a function `pval_sign`, which has the same input as `sign_test`, but with the null median *first*. Run it on your data and see what it gives.

Solution: Try it and see:

```
pval_sign(60, times, time)

## [1] 0.0266037
```

The two-sided P-value, and that is all. We'll be using this in a minute.

Alternatively, there is also this, which needs a null median and a *column* as input:

```
pval_sign0(60, times$time)

## [1] 0.0266037
```

- (e) Obtain a 95% confidence interval for the median based on these data. Do this two ways. First, use the trial and error way from class (either the try-lots-of-values way or the bisection way; either is good). Second, use `ci_median` from `smmr`. The latter takes as input a data frame, a column name (unquoted) and optionally a `conf.level` that defaults to 0.95.

Solution: The reason for showing you `pval_sign` in the previous part is that this is a building block for the confidence interval.

What we do is to try various null medians and find out which ones give P-values less than 0.05 (outside the interval) and which ones bigger (inside). We know that the value 60 is outside the 95% CI, and the sample median is close to 50 (which we expect to be inside), so sensible values to try for the upper end of the interval would be between 50 and 60:

```
pval_sign(58, times, time)

## [1] 0.0266037

pval_sign(55, times, time)

## [1] 0.6636238
```

So, 55 is inside the interval and 58 is outside. I could investigate further in similar fashion, but I thought I would try a whole bunch of null medians all at once. That goes like this:

```
meds=seq(55,58,0.25)
meds

## [1] 55.00 55.25 55.50 55.75 56.00 56.25 56.50 56.75 57.00 57.25 57.50
## [12] 57.75 58.00

pvals=map_dbl(meds,pval_sign,times,time)
data.frame(meds,pvals)

##      meds      pvals
## 1  55.00 0.66362381
## 2  55.25 0.38331032
## 3  55.50 0.26317596
## 4  55.75 0.18924713
## 5  56.00 0.18924713
## 6  56.25 0.18924713
## 7  56.50 0.07835388
## 8  56.75 0.07835388
## 9  57.00 0.07835388
## 10 57.25 0.07835388
## 11 57.50 0.07835388
## 12 57.75 0.02660370
## 13 58.00 0.02660370
```

So values for the median all the way up to and including 57.5 are in the confidence interval.

What `map_dbl` does is to take a vector of values, here the ones in `meds` (55 through 58 in steps of 0.25), feed them into a function, here `pval_sign`, one by one and gather together the results. `pval_sign` has two other inputs, `times` and `time`, which are added to `map_dbl` at the end. (They are the same no matter what median we are testing.) So, putting the calculated P-values side by side with the null medians they belong to shows you which medians are inside the confidence interval and which are outside.

The function is called `map_dbl` because *my* function called `pval_sign` that is called repeatedly returns a single decimal number (a `dbl`). There is also, for example, `map_chr` for repeatedly calling a function that returns a single piece of text, and plain `map` that is used when the repeatedly-called function returns a data frame.

Since you don't know about `map_dbl`, I didn't want to confuse things more than necessary, but now that you *do* know what it does, you might be in a better position to understand this more Tidyverse-flavoured code, with the `map_dbl` inside a `mutate` and the data frame created as we go:

```
tibble(meds=seq(55,58,0.25)) %>%
  mutate(pvals=map_dbl(meds,pval_sign,times,time))

## # A tibble: 13 x 2
##   meds  pvals
##   <dbl> <dbl>
## 1  55    0.664
## 2  55.2  0.383
## 3  55.5  0.263
## 4  55.8  0.189
## 5  56    0.189
## 6  56.2  0.189
## 7  56.5  0.0784
## 8  56.8  0.0784
## 9  57    0.0784
## 10 57.2  0.0784
## 11 57.5  0.0784
## 12 57.8  0.0266
## 13 58    0.0266
```

Now for the other end of the interval. I'm going to do this a different way: more efficient, but less transparent. The first thing I need is a pair of values for the median: one inside the interval and one outside. Let's try 40 and 50:

```
pval_sign(40,times,time)
## [1] 0.00719738

pval_sign(50,times,time)
## [1] 1
```

OK, so 40 is outside and 50 is inside. So what do I guess for the next value to try? I could do something clever like assuming that the relationship between hypothesized median and P-value is *linear*, and then guessing where that line crosses 0.05. But I'm going to assume *nothing* about the relationship except that it goes uphill, and therefore crosses 0.05 somewhere. So my next guess is halfway between the two values I tried before:

```
pval_sign(45,times,time)
## [1] 0.07835388
```

So, 45 is inside the interval, and my (slightly) improved guess at the bottom end of the interval is that it's between 40 and 45. So next, I try halfway between *those*:

```
pval_sign(42.5,times,time)
## [1] 0.0266037
```

42.5 is outside, so the bottom end of the interval is between 42.5 and 45.

What we are doing is narrowing down where the interval's bottom end is. We started by knowing it to within 10, and now we know it to within 2.5. So if we keep going, we'll know it as accurately as we wish.

This is called a "bisection" method, because at each step, we're dividing our interval by 2.

There is one piece of decision-making at each step: if the P-value for the median you try is

greater than 0.05, that becomes the top end of your interval (as when we tried 45); if it is less, it becomes the bottom end (when we tried 42.5).

This all begs to be automated into a loop. It's not a **for**-type loop, because we don't know how many times we'll be going around. It's a **while** loop: keep going while something is true. Here's how it goes:

```
lo=40
hi=50
while(abs(hi-lo)>0.1) {
  try=(hi+lo)/2
  ptry=pval_sign(try,times,time)
  print(c(try,ptry))
  if (ptry<0.05) {
    lo=try
  } else {
    hi=try
  }
}

## [1] 45.00000000  0.07835388
## [1] 42.50000000  0.0266037
## [1] 43.75000000  0.0266037
## [1] 44.37500000  0.07835388
## [1] 44.06250000  0.0266037
## [1] 44.21875000  0.0266037
## [1] 44.29687500  0.0266037

lo

## [1] 44.29688

pval_sign(lo,times,time)

## [1] 0.0266037

hi

## [1] 44.375

pval_sign(hi,times,time)

## [1] 0.07835388
```

The loop stopped because 44.297 and 44.375 are less than 0.1 apart. The first of those is outside the interval and the second is inside. So the bottom end of our interval is 44.375, to this accuracy. If you want it more accurately, change 0.1 in the **while** line to something smaller (but then you'll be waiting longer for the answer).

I put the **print** statement in the loop so that you could see what values were being tried, and what P-values they were producing. What happens with these is that the P-value jumps at each data value, so you won't get a P-value exactly 0.05; you'll get one above and one below.

Likewise, you can use the function with a zero on its name and feed it a column rather than a data frame and a column name:

```

meds=seq(55,58,0.25)
meds

## [1] 55.00 55.25 55.50 55.75 56.00 56.25 56.50 56.75 57.00 57.25 57.50
## [12] 57.75 58.00

pvals=map_dbl(meds,pval_sign0,times$time)
data.frame(meds,pvals)

##      meds      pvals
## 1  55.00 0.66362381
## 2  55.25 0.38331032
## 3  55.50 0.26317596
## 4  55.75 0.18924713
## 5  56.00 0.18924713
## 6  56.25 0.18924713
## 7  56.50 0.07835388
## 8  56.75 0.07835388
## 9  57.00 0.07835388
## 10 57.25 0.07835388
## 11 57.50 0.07835388
## 12 57.75 0.02660370
## 13 58.00 0.02660370

```

Or adapt the idea I had above for bisection.

All that was a lot of work, but I wanted you to see it all once, so that you know where the confidence interval is coming from. `smmr` also has a function `ci_median` that does all of the above without you having to do it. As I first wrote it, it was using the trial and error thing with `map_dbl`, but I chose to rewrite it with the bisection idea, because I thought that would be more accurate.

```

ci_median(times,time)

## [1] 44.30747 57.59766

```

This is a more accurate interval than we got above. (The `while` loop for the bisection keeps going until the two guesses at the appropriate end of the interval are less than 0.01 apart, by default.)¹

If you want some other confidence level, you add `conf.level` on the end, as you would for `t.test`:

```

ci_median(times,time,conf.level=0.75)

## [1] 46.20444 55.49473

```

A 75% CI, just for fun. This is a shorter interval than the 95% one, as it should be.

Likewise there is a `ci_median0` that takes a column and an optional confidence level:

```

ci_median0(times$time)

## [1] 44.30747 57.59766

ci_median0(times$time,conf.level=0.75)

## [1] 46.20444 55.49473

```

with the same results. Try `ci_median` first, and if it doesn't work, try `ci_median0`.

2. A famous cookie manufacturer claims that their bags of chocolate chip cookies contain “more than 1100 chocolate chips on average”. A diligent group of students buys 16 bags of these cookies and counts the number of chocolate chips in each bag. The results are in <http://www.utsc.utoronto.ca/~butler/c32/chips.txt>.

(a) Read in and display (some of) the data.

Solution: I'll pretend it's a `.csv` this time, just for fun. Give the data frame a name different from `chips`, so that you don't get confused:

```
bags=read_csv("chips.txt")
## Parsed with column specification:
## cols(
##   chips = col_integer()
## )
bags
## # A tibble: 16 x 1
##   chips
##   <int>
## 1  1219
## 2  1214
## 3  1087
## 4  1200
## 5  1419
## 6  1121
## 7  1325
## 8  1345
## 9  1244
## 10 1258
## 11 1356
## 12 1132
## 13 1191
## 14 1270
## 15 1295
## 16 1135
```

That looks sensible.

- (b) Build your own sign test in R for testing that the median is 1100 chocolate chips, against the alternative that it is greater. (Do this as in class: count the appropriate thing, compare it with an appropriate binomial distribution, and obtain a P-value.

Solution: The null median is 1100, so we count the number of values above and below:


```
bags %>% count(chips<1100)

## # A tibble: 2 x 2
##   `chips < 1100`      n
##   <lgl>          <int>
## 1 FALSE          15
## 2 TRUE           1
```

The un-standard thing there is that we can put a logical condition directly into the `count`. If you don't think of that, you can also do this, which creates a new variable `less` that is `TRUE` or `FALSE` for each bag appropriately:

```
bags %>% mutate(less=(chips<1100)) %>% count(less)

## # A tibble: 2 x 2
##   less      n
##   <lgl> <int>
## 1 FALSE   15
## 2 TRUE    1
```

or the more verbose

```
bags %>% mutate(less=(chips<1100)) %>%
  group_by(less) %>% summarize(howmany=n())

## # A tibble: 2 x 2
##   less howmany
##   <lgl>   <int>
## 1 FALSE     15
## 2 TRUE      1
```

Just one value below, with all the rest above.

Getting the right P-value, properly, requires some careful thought (but you will probably get the right answer anyway). If the alternative hypothesis is true, and the median is actually bigger than 1100 (say, 1200), you would expect half the data values to be bigger than 1200 and half smaller. So *more* than half the data values would be bigger than 1100, and fewer than half of them would be less than 1100. So, if we are going to reject the null (as it looks as if we will), that small number of values below 1100 is what we want.

The P-value is the probability of a value 1 or less in a binomial distribution with $n = 16, p = 0.5$:

```
sum(dbinom(0:1,16,0.5))

## [1] 0.0002593994
```

Or, equivalently, count *up* from 15:

```
sum(dbinom(15:16,16,0.5))

## [1] 0.0002593994
```

This is correctly one-sided, so we don't have to do anything with it.

- (c) Use my R package `smmr` to reproduce your sign test above, and verify that you get consistent results. (See the maze-design question for instructions on installing this, if you haven't yet.)

Solution: This will mean reading the output carefully:

```
library(smmr)
sign_test(bags, chips, 1100)

## $above_below
## below above
##      1      15
##
## $p_values
##   alternative      p_value
## 1      lower 0.9999847412
## 2      upper 0.0002593994
## 3 two-sided 0.0005187988
```

This time, we're doing a one-sided test, specifically an *upper-tail* test, since we are looking for evidence that the median is *greater than* 1100. The results are exactly what we got “by hand”: 15 values above and one below, and a P-value (look along the **upper** line) of 0.00026.

Alternatively, you can do this:

```
sign_test0(bags$chips, 1100)

## $above_below
## below above
##      1      15
##
## $p_values
##   alternative      p_value
## 1      lower 0.9999847412
## 2      upper 0.0002593994
## 3 two-sided 0.0005187988
```

with the same result (but only go this way if you need to).

- (d) Use `smmr` to obtain a 95% confidence interval for the median number of chocolate chips per bag of cookies.

Solution: Once everything is in place, this is simplicity itself:

```
ci_median(bags, chips)

## [1] 1135.003 1324.996
```

1135 to 1325. I would round these off to whole numbers, since the data values are all whole numbers. These values are all above 1100, which supports the conclusion we got above that the median is above 1100. This is as it should be, because the CI is “all those medians that would *not* be rejected by the sign test”.

Or,

```
ci_median0(bags$chips)

## [1] 1135.003 1324.996
```

3. I've mentioned several times that the sign test has less power than the *t*-test. Let's investigate this with a specific example.

Let's suppose we are testing $H_0 : \mu = 40$ against $H_a : \mu \neq 40$, where μ is the population mean (and median, as we shall see). Our population actually has a normal distribution with mean 50 and SD 15, so that the null hypothesis is *wrong* and we want to reject it most of the time. On the other hand, the population actually *is* normally-distributed and so the *t*-test is the right one to use.

(This is an old question, so I tackle the simulated power differently than I did it in class this time. But see if you can follow what I do here.)

- (a) Use `power.t.test` to find the probability that a *t*-test correctly rejects the null hypothesis using a sample size of $n = 10$.

Solution:

```
power.t.test(delta=50-40,n=10,sd=15,type="one.sample",alternative="two.sided")

##
##      One-sample t test power calculation
##
##              n = 10
##            delta = 10
##             sd = 15
##    sig.level = 0.05
##      power = 0.4691805
##    alternative = two.sided
```

The power is 0.469. Not great, but we'll see how this stacks up against the sign test.

- (b) What code in R would draw a random sample of size 10 from the *true* population distribution and save the sample in a variable?

Solution: The data actually have a normal distribution with mean 50 and SD 15, so we use `rnorm` with this mean and SD, obtaining 10 values:

```
x=rnorm(10,50,15)
x
## [1] 38.48692 65.12505 49.28490 65.47715 49.21718 53.28321 75.41723
## [8] 43.11818 56.58732 50.02373
```

- (c) What code would count how many of the sampled values are less than 40 and how many are greater (or equal)?

Solution: The way we know this is to put `x` into a data frame first:

```
tibble(x) %>% count(x<40)

## # A tibble: 2 x 2
##   `x < 40`     n
##   <lgl>      <int>
## 1 FALSE         9
## 2 TRUE          1
```

2 values less (and 8 greater-or-equal).

- (d) It turns out the sign test would reject $H_0 : M = 40$ against $H_a : M \neq 40$ at $\alpha = 0.05$ if the smaller

of the numbers in the last part is 1 or less. (M is the population median.) Add to your pipeline to obtain **TRUE** if you should reject the null for your data and **FALSE** otherwise.

Solution: This is actually easier than you might think. The output from `count` is a data frame with a column called `n`, whose minimum value you want. I add to my pipeline:

```
tibble(x) %>% count(x<40) %>%
  summarize(the_min=min(n)) %>%
  mutate(is_rejected=(the_min<=1))

## # A tibble: 1 x 2
##   the_min is_rejected
##   <dbl> <lgl>
## 1      1 TRUE
```

This will fail sometimes. If all 10 of your sample values are greater than 40, which they might turn out to be, you'll get a table with only one line, **FALSE** and 10; the minimum of the `n` values is 10 (since there is only one), and it will falsely say that you should not reject. The fix is

```
tibble(x) %>% count(x<40) %>%
  summarize(the_min=min(n)) %>%
  mutate(is_rejected=(the_min<=1 | the_min==10))

## # A tibble: 1 x 2
##   the_min is_rejected
##   <dbl> <lgl>
## 1      1 TRUE
```

The above is almost the right thing, but not quite: we only want that value that I called **is_rejected**, rather than the whole data frame, so a `pull` will grab it:

```
tibble(x) %>% count(x<40) %>%
  summarize(the_min=min(n)) %>%
  mutate(is_rejected=(the_min<=1 | the_min==10)) %>%
  pull(is_rejected)

## [1] TRUE
```

You might be wondering where the “1 or less” came from. Getting a P-value for the sign test involves the binomial distribution: if the null is correct, each data value is independently either above or below 40, with probability 0.5 of each, so the number of values below 40 (say) is binomial with $n = 10$ and $p = 0.5$. The P-value for 1 observed value below 40 and the rest above is

```
2*pbinom(1,10,0.5)

## [1] 0.02148438
```

which is less than 0.05; the P-value for 2 values below 40 and the rest above is

```
2*pbinom(2,10,0.5)

## [1] 0.109375
```

which is bigger than 0.05.

You might have encountered the term “critical region” for a test. This is the values of your test statistic that you would reject the null hypothesis for. In this case, the critical region is 1 and 0 observations below 40, along with 1 and 0 observations above 40.

When you're thinking about power, I think it's easiest to think in terms of the critical region (rather than directly in terms of P-values) since you have a certain α in mind all the way through, 0.05 in the power examples that I've done. The steps are then:

- Work out the critical region for your test, the values of the test statistic (or sample mean or sample count) that would lead to rejecting the null hypothesis.
- Under your particular alternative hypothesis, find the probability of falling into your critical region.

When I say “work out”, I mean either calculating (along the lines of STAB57), or simulating, as we have done here.

- (e) Use `rerun` to simulate the above process 1000 times: drawing a random sample from a normal distribution with mean 50 and SD 15, counting the number of values below and above 40, rejecting if the minimum of those is 1 or less, then counting the number of rejections out of 1000.

Solution: The way we've used `rerun` is to use it to select the random samples, and then we use `map` ideas to do what we want to do with each random sample, along the lines of what we did with the one random sample above.

This is liable to go wrong the first few times, so make sure that each line works before you go on to the next. (`rerun` will produce you a `list` of random samples, with each of which you want to do something.) While you're debugging, try it with a small number of random samples like 5.

I start with setting the random number seed, so it comes out the same each time. I discuss the results below and the code below that.

```
set.seed(457299)
rerun(1000, rnorm(10, 50, 15)) %>%
  map(~tibble(x=)) %>%
  map(~count(., x<40)) %>%
  map(~summarize(., the_min=min(n))) %>%
  map(~mutate(., is_rejected=(the_min<=1 | the_min==10))) %>%
  map_lgl(~pull(., is_rejected)) %>%
  tibble(was_true=.) %>%
  count(was_true)

## # A tibble: 2 x 2
##   was_true     n
##   <lgl>     <int>
## 1 FALSE     757
## 2 TRUE      243
```

The estimated power of the sign test is 0.243, since that was the number of times a simulated sample gave us 0 or 1 values above or below 40 (and the rest on the other side).

All right, that code is seriously scary. Let me try to take you through it.

- The `rerun` line is the same kind of thing we had before: generate 1000 different random samples of size 10 from a normal distribution with mean 50 and SD 15.
- The output from the previous step is a `list` of vectors. But we like data frames to count things in, so for each vector we turn it into a data frame, filling a column called `x` with

whatever was in each vector (that random sample). So now we have 1000 data frames each containing a column called `x`.

- Next, in each of those data frames, count up how many of the `x` values are less than 40. This will produce a data frame each time containing a column `n` that is the frequencies. Here and below, the `.` is used to denote “it”: that is, each of the elements of the list created originally by `rerun` that we are doing something with. Also, at the moment, the output for each element of the list is a data frame, so we stick with `map` for the time being.
- Next, for each of those tables of frequencies, find the smallest one and call it `the_min`. (As discussed above, all the values might be bigger than 40, in which case `the_min` will be 10 rather than 0, which we handle next.)
- Next, we create a new column called `is_rejected` which says that we should reject a median of 40 if the minimum value we just calculated is 1 or less, or if it happens to be 10, in which case that would have been the only entry in the frequency table, so that the missing one would have been zero.
- Next, we pull out only the true-or-false value in `is_rejected`. At last, the answer here is not a data frame but a simple logical value; `map_lgl` is like `map_dbl` except that the thing we are doing returns a `TRUE` or a `FALSE` rather than a number.
- At this point we have a vector of 1000 true-or-false. We want to count them, so we put them into a data frame (with a column called `was_true`), and in the last line, count them up. There are 243 (correct) rejections and 757 (incorrect) non-rejections.

You may now breathe again.

I’m now thinking a better way to do this is to write a function that takes a sample (in a vector) and returns a `TRUE` or `FALSE` according to whether or not a median of 40 would be rejected for that sample:

```
is_reject=function(x) {  
  tibble(x=x) %>%  
    count(x<40) %>%  
    summarize(the_min=min(n)) %>%  
    mutate(is_rejected=the_min<=1 | the_min==10) %>%  
    pull(is_rejected)  
}
```

Now, we have to use that. This function will be “mapped” over for each of the random samples that come out of `rerun`, but now there will be only one `map` because the complication of the multiple `maps` has been subsumed into this one function. I’ll set my random number seed so that I get the same results as before:

```
set.seed(457299)  
rerun(1000, rnorm(10, 50, 15)) %>%  
  map_lgl(~is_reject(.)) %>%  
  tibble(rejected=.) %>%  
  count(rejected)  
  
## # A tibble: 2 x 2  
##   rejected     n  
##   <lgl>    <int>  
## 1 FALSE     757  
## 2 TRUE      243
```

Same results, and yeah, I like that a lot better.

(f) Which is more powerful in this case, the sign test or the t -test? How do you know?

Solution: The power of the sign test is estimated as 0.243, which is quite a bit less than the power of the t -test, which we found back in (a) to be 0.469. So the t -test, in this situation where it is valid, is the right test to use: it is (a) valid and (b) more powerful.

So the t -test is more powerful. One way to think about how *much* more powerful is to ask “how much smaller of a sample size would be needed for the t -test to have the same power as this sign test?” The power of my sign test was 0.243, so in `power.t.test` we set `power` equal to that and omit the sample size `n`:

```
power.t.test(delta=50-40,power=0.243,sd=15,type="one.sample",alternative="two.sided")

##
##      One-sample t test power calculation
##
##              n = 5.599293
##            delta = 10
##              sd = 15
##      sig.level = 0.05
##            power = 0.243
##      alternative = two.sided
```

A sample of size 6 gives the same power for the t -test that a sample of size 10 does for the sign test. The ratio of these two sample sizes is called the *relative efficiency* of the two tests: in this case, the t -test is $10/6 = 1.67$ times more efficient. The data that you have are being used “more efficiently” by the t -test.

It is possible to derive² the limiting relative efficiency of the t test relative to the sign test when the data are actually normal, as the sample size gets larger. This turns out not to depend on how far wrong the null is (as long as it is the same for both the t -test and the sign test). This “asymptotic relative efficiency” is $\pi/2 = 1.57$. Our relative efficiency for power 0.243, namely 1.67, was pretty close to this, even though our sample sizes 10 and 6 are not especially close to infinity.

This says that, if your data are actually from a normal distribution, you do a lot better to use the t -test than the sign test, because the sign test is wasteful of data (it only uses above/below rather than the actual values).

If your data are *not* from a normal distribution, then the story can be very different.

Of course you knew I would investigate this. There is a distribution called the “Laplace” or “double exponential” distribution, that has very long tails.³ The distribution is not in base R, but there is a package called `smoothest` that contains a function `rdoublex` to generate random values from this distribution. So we’re going to do a simulation investigation of the power of the sign test for Laplace data, by the same simulation technique that we did above. Like the normal, the Laplace distribution is symmetric, so its mean and median are the same (which makes our life easier).⁴

Let’s test the hypothesis that the median is zero. We’ll suppose that the true median is 0.5 (this is called `mu` in `rdoublex`). The first problem we run into is that we can’t use `power.t.test` because they assume normal data, which we are far from having. So we have to do two

simulations: one to simulate the power of the t test, and one to simulate the power of the sign test.

To simulate the t test, we first have to generate some Laplace data with the true mean of 0.5. We'll use a sample size of 50 throughout these simulations.

```
library(smoothest)

## Loading required package: MASS

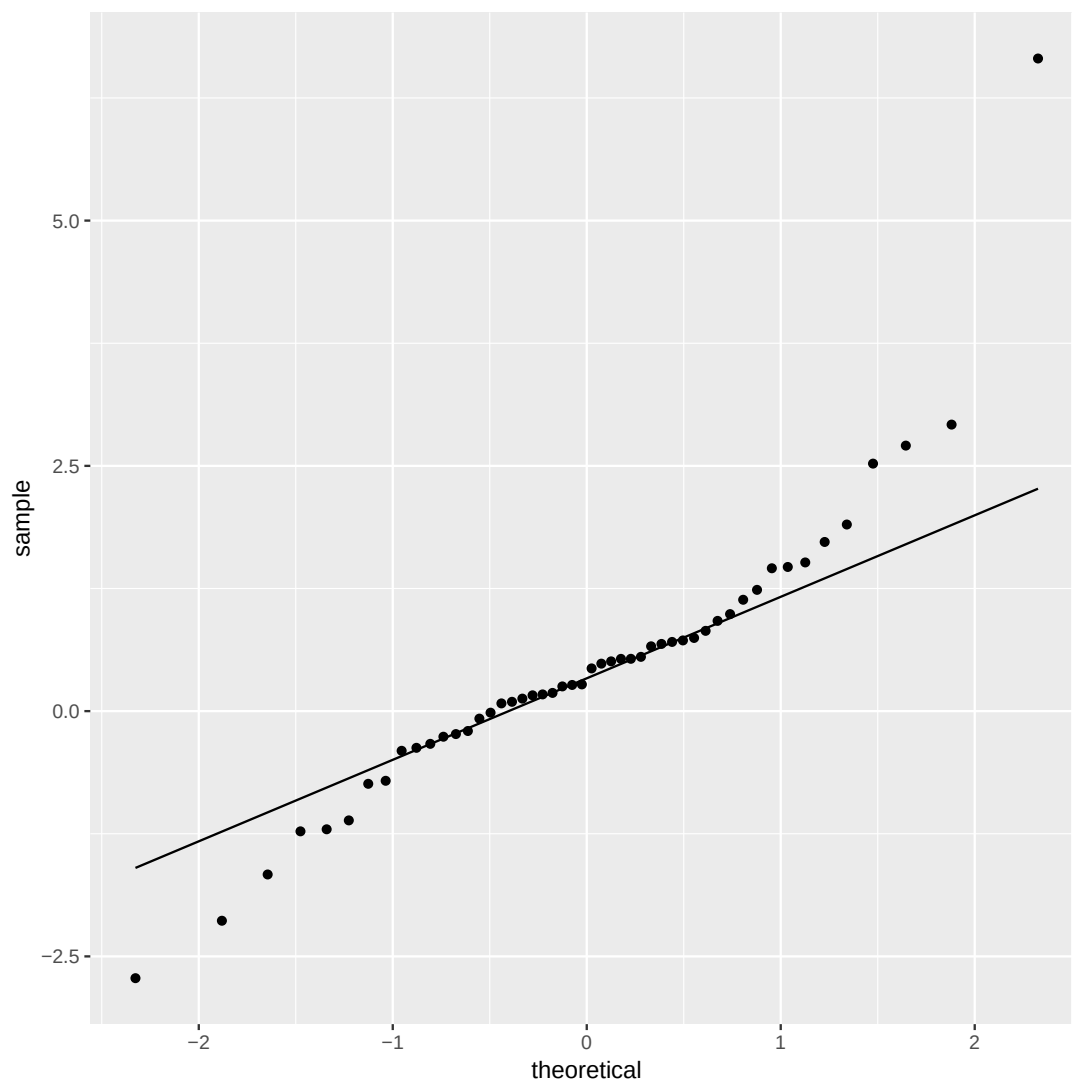
##
## Attaching package: 'MASS'

## The following object is masked from 'package:dplyr':
##
## select

rl=rdoublex(50,mu=0.5)
rl
## [1] -0.33323285  0.70569291 -1.22513053  0.68517708  0.12778518
## [6]  0.50749949  0.26700527  1.90236874  0.53288312 -0.37374732
## [11]  0.27256566  0.53365929  0.43581431 -0.01545866  0.18594908
## [16] -0.40403202  1.13540289  0.16137306 -0.23360644 -0.74050354
## [21]  2.92089551 -2.72173880  0.48428815  1.23636045  0.17078618
## [26]  1.72456334  0.07903058  0.25210411  0.09512810  2.52310082
## [31] -2.13629814  0.81851434  0.74615575 -0.26068744  2.70683355
## [36]  1.46981530  1.45646489 -0.20232517  6.65249860  1.51575026
## [41] -0.07606399 -1.11338640 -1.20427995 -0.70986104 -1.66466321
## [46]  0.55346854  0.66091469  0.72100677  0.92025176  0.98922656
```

This seems to have some unusual values, far away from zero:

```
tibble(rl) %>%
  ggplot(aes(sample=rl))+
  stat_qq()+stat_qq_line()
```

You see the long tails compared to the normal.

Now, we feed these values into `t.test` and see whether we reject a null median of zero (at $\alpha = 0.05$):

```

tt=t.test(r1)
tt

##
## One Sample t-test
##
## data:  r1
## t = 2.2556, df = 49, p-value = 0.0286
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  0.04959344 0.85981931
## sample estimates:
## mean of x
## 0.4547064

```

Or we can just pull out the P-value and even compare it to 0.05:

```

pval=tt$p.value
pval

## [1] 0.02859596

is.reject=(pval<=0.05)
is.reject

## [1] TRUE

```

This one has a small P-value and so the null median of 0 should be (correctly) rejected.

We'll use these ideas to simulate the power of the *t*-test for these data, testing a mean of 0. This uses the same ideas as for any power simulation; the difference here is the true distribution:

```

rerun(1000,rdoublex(50,mu=0.5)) %>%
  map(~t.test(.,mu=0)) %>%
  map_dbl("p.value") ->
pvals

```

and then count them:

```

tibble(pvals) %>% count(pvals<=0.05)

## # A tibble: 2 x 2
##   `pvals <= 0.05`     n
##   <lgl>           <int>
## 1 FALSE           304
## 2 TRUE            696

```

And now we simulate the sign test. Since what we want is a P-value from a vector, the easiest way to do this is to use `pval.sign0` from `smmr`, which returns exactly the two-sided P-value that we want, so that the procedure is a step simpler:

```

rerun(1000,rdoublex(50,mu=0.5)) %>%
  map_dbl(~pval_sign0(0,.)) ->
pvals_sign

```

and then

```
tibble(pvals_sign) %>% count(pvals_sign<=0.05)

## # A tibble: 2 x 2
##   `pvals_sign <= 0.05`      n
##   <lgl>                <int>
## 1 FALSE                239
## 2 TRUE                 761
```

For data from this Laplace distribution, the power of this t -test is 0.696, but the power of the sign test on the same data is 0.761, *bigger*. For Laplace-distributed data, the sign test is *more* powerful than the t -test.

This is not to say that you will ever run into data that comes from the Laplace distribution. But the moral of the story is that the sign test *can* be more powerful than the t -test, under the right circumstances (and the above simulation is the “proof” of that statement). So a blanket statement like “the sign test is not very powerful” needs to be qualified a bit: when your data come from a sufficiently long-tailed distribution, the sign test can be more powerful relative to the t -test than you would think.

I finish by “unloading” the two packages that got loaded:

```
detach(package:smoothest, unload=T)
detach(package:MASS, unload=T)
```

4. Athletes are concerned with measuring their body fat percentage. Two different methods are available: one using ultrasound, and the other using X-ray technology. We are interested in whether there is a difference in the mean body fat percentage as measured by these two methods, and if so, how big that difference is. Data on 16 athletes are at <http://www.utsc.utoronto.ca/~butler/c32/bodyfat.txt>.
 - (a) Explain briefly why a matched pairs analysis is more suitable for these data than a two-independent-samples analysis (using a two-sample t -test). You might find that looking at the data (clicking on the link) helps you figure this out.

Solution: The data file looks like this:

```
athlete xray ultrasound
1 5.00 4.75
2 7 3.75
3 9.25 9
4 12 11.75
5 17.25 17
6 29.5 27.5
7 5.5 6.5
8 6 6.75
9 8 8.75
10 8.5 9.5
11 9.25 9.5
12 11 12
13 12 12.25
14 14 15.5
15 17 18
16 18 18.25
```

The data are two measurements for each of the 16 athletes: that is, each athlete had their body

fat percentage measured using *both* of the two methods.

Extra: a two-sample t approach would be reasonable if one set of 16 athletes had been measured by X-ray and *another different* set of 16 athletes had been measured by ultrasound. (That is, if there had been 32 athletes in total, with each one randomly assigned to *one* of the measurement methods.) But that's not what happened. It is easy to measure one athlete's body fat percentage using both of the two methods, so a matched pairs design is easy to implement (as well as being better). If you use two independent samples (each athlete doing only one measurement method), you introduce an extra source of variability: athletes differ one from another in body fat, as well as differing possibly by measurement method. If you use a matched-pairs design, you remove the athlete-to-athlete differences, leaving only the differences due to measurement method.

- (b) Read in the data and check that you have a sensible number of rows and columns.

Solution: This kind of thing. Since you looked at the data (didn't you?), you'll know that the values are separated by single spaces:

```
myurl="http://www.utsc.utoronto.ca/~butler/c32/bodyfat.txt"
bodyfat=read_delim(myurl," ")
```

```
## Parsed with column specification:
## cols(
##   athlete = col_integer(),
##   xray = col_double(),
##   ultrasound = col_double()
## )
```

```
bodyfat
```

```
## # A tibble: 16 x 3
##   athlete xray ultrasound
##   <int> <dbl>     <dbl>
## 1      1  1 5      4.75
## 2      2  2 7      3.75
## 3      3  3 9.25    9
## 4      4  4 12     11.8
## 5      5  5 17.2    17
## 6      6  6 29.5    27.5
## 7      7  7 5.5     6.5
## 8      8  8 6      6.75
## 9      9  9 8      8.75
## 10     10 10 8.5     9.5
## 11     11 11 9.25    9.5
## 12     12 12 11     12
## 13     13 13 12     12.2
## 14     14 14 14     15.5
## 15     15 15 17     18
## 16     16 16 18     18.2
```

16 rows (athletes) and 3 columns, one for each measurement method and one labelling the athletes. All good.

Since 16 is not too much bigger than 10, I got the whole data frame here. (At least, I think

that's the reason I got more than 10 rows.) In an R Notebook, you'll see the first ten rows as normal, with a button to click to see the other six.

- (c) Carry out a suitable test to determine whether the means are the same or different. (At this point, obtain the R output including a P-value.)

Solution: Feed the two columns into `t.test` along with `paired=T`. This is a two-sided test, so we don't have to take any special steps for that. Note that we're back to the "old-fashioned" version of `t.test` that *does not* allow `data=`, so we have to go the `with` way:

```
with(bodyfat,t.test(xray,ultrasound,paired=T))

##
## Paired t-test
##
## data: xray and ultrasound
## t = -0.30801, df = 15, p-value = 0.7623
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -0.7425068 0.5550068
## sample estimates:
## mean of the differences
## -0.09375
```

- (d) What do you conclude from the test?

Solution: The P-value of 0.7623 is not at all small, so there is no way we can reject the null hypothesis.⁵ There is no evidence of a difference in means; we can act as if the two methods produce the same mean body fat percentage.

That is to say, on this evidence we can use either method, whichever one is cheaper or more convenient.

Extra: an informal way of seeing whether this is reasonable is to see how often the first value of the pair is larger, and how often the second. If the null hypothesis is true, there should be about a 50–50 split, but if false, there should be a very uneven split. Here that would go like this:

```
with(bodyfat,table(xray>ultrasound))

##
## FALSE TRUE
## 10 6
```

or like this:

```
bodyfat %>% count(xray>ultrasound)

## # A tibble: 2 x 2
##   `xray > ultrasound`     n
##   <lg1>              <int>
## 1 FALSE              10
## 2 TRUE               6
```

6 times the X-ray value was bigger, and 10 times the ultrasound value was. A pretty even split, so it's not surprising that we failed to reject the null. (You might recognize this as an informal version of the sign test.)

- (e) Obtain a 95% confidence interval for the population mean difference. How is the interval consistent with your test?

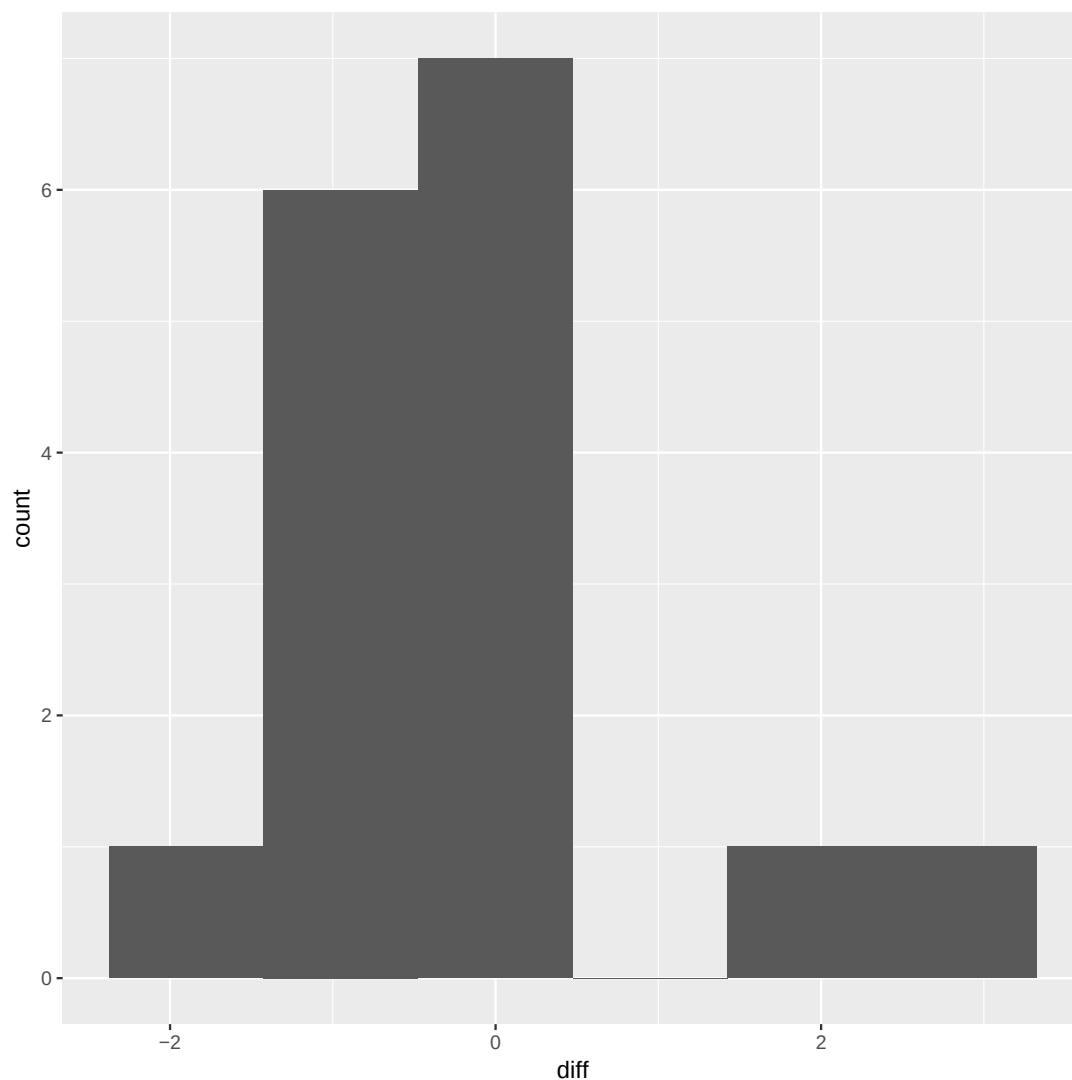
Solution: You don't even need to do any more coding: the test was two-sided, so just pick the confidence interval off the output above: -0.74 to 0.56 .

The interval includes both positive and negative values (or, 0 is inside the interval), so the difference could go either way. This is entirely consistent with not being able to reject the null.

- (f) Calculate the differences, and make a normal quantile plot of them. Is there any evidence that normality of differences fails? Explain briefly. (If we haven't gotten to normal quantile plots in class when you read this, make a different plot to assess this issue.)

Solution: The smoothest⁶ way to do this is to use a pipeline: use a `mutate` to create the column of differences, and then pipe that into `ggplot`, omitting the data frame that would normally go first (the input data frame here is the new one with the differences in it, which doesn't have a name). I'll make a normal quantile plot in a moment, but if we haven't talked about that yet, the plot to make is a histogram:

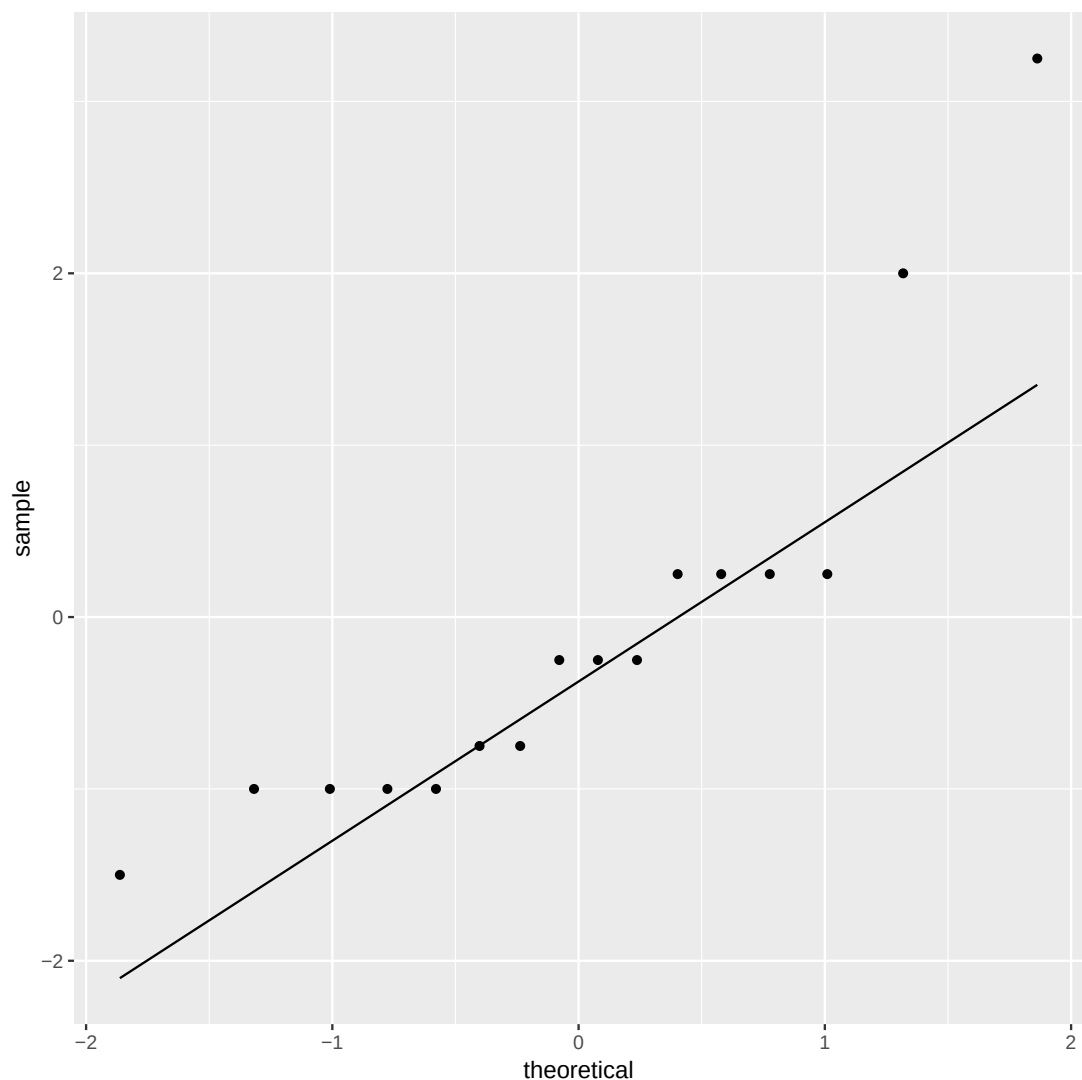
```
bodyfat %>% mutate(diff=xray-ultrasound) %>%  
  ggplot(aes(x=diff))+geom_histogram(bins=6)
```



I don't know whether you'd call that "approximately normal" or not. We are in kind of a double-bind with this one: the sample size is small, so normality matters, but with a small sample, the data might not look very normal. It's kind of skewed right, but most of the evidence for the skewness is contained in those two observations with difference 2 and above, which is pretty flimsy evidence for anything. (In the normal quantile plot below, the suggestion is that those two observations really are a bit too large. It's easier to tell there.)

Below, I'm repeating the calculation of the differences, which is inefficient. If I'm going to draw two graphs of the differences, the right way is to calculate the differences *and save the data frame*, then use that new data frame twice. But you're probably only going to draw either the histogram or the normal quantile plot, not both, so you can use the appropriate one of my two bits of code. The normal quantile plot:

```
bodyfat %>% mutate(diff=xray-ultrasound) %>%
  ggplot(aes(sample=diff))+stat_qq()+stat_qq_line()
```



This is showing a little evidence of skewness or outliers (depending on your point of view: either is good). The lowest and highest values are both too high, and the pattern of points on the plot is kind of curved (which would be evidence of skewness). Or you could say that the two highest values are too high, with the other values being more or less in line (that would be evidence of outliers at the upper end). I like outliers better than skewness, since those bottom-end points are not far off the line. I would also accept “no substantial problems”, if you can make the case that those two highest points are not too far off the line. With only 16 observations as we have here, even truly normal data would stray off the line a bit.

As ever, your explanation is more important than your conclusion. Can you justify what you think?

If you took your differences the other way around, as `ultrasound` minus `xray`, your plot will also be the other way around, with the “outliers” at the bottom. That’s good too.

Where this is going (which I didn’t ask you) is whether or not we trust the result of the matched pairs test. I would say that the test is so far from being significant, and the failure of normality

is not gross, that it is hard to imagine *any* alternative test coming up with a significant result. So I would be happy to trust this paired *t*-test.

5. Can students throw a baseball farther than a softball? A statistics class, containing 24 students, went out to a football field to try to answer this question. Each student warmed up and then threw each type of ball as far as they could. The order of ball types was randomized: some students threw the baseball first, and some threw the softball first. (A softball is bigger than a baseball, so we might expect that a softball would be harder to throw a long way than a baseball.) The data are in <http://www.utsc.utoronto.ca/~butler/c32/throw.txt> in three columns: the first is a number identifying the student, the second is the distance thrown with the baseball (in yards) and the third is the distance thrown with the softball (also in yards).

- (a) Read the data into R. You'll need to supply some names to the columns.

Solution: This kind of thing:

```
myurl="http://www.utsc.utoronto.ca/~butler/c32/throw.txt"
throws=read_delim(myurl," ",col_names=c("student","baseball","softball"))

## Parsed with column specification:
## cols(
##   student = col_integer(),
##   baseball = col_integer(),
##   softball = col_integer()
## )

throws

## # A tibble: 24 x 3
##   student baseball softball
##   <int>      <int>      <int>
## 1         1         65         57
## 2         2         90         58
## 3         3         75         66
## 4         4         73         61
## 5         5         79         65
## 6         6         68         56
## 7         7         58         53
## 8         8         41         41
## 9         9         56         44
## 10        10         70         65
## # ... with 14 more rows
```

This is one of those times where we have to tell R what names to give the columns. Or you can put `col_names=F` and leave the columns called `X1`, `X2`, `X3` or whatever they end up as.

- (b) Calculate a column of differences, baseball minus softball, *in* the data frame.

Solution: Add it to the data frame using `mutate`. Use the right-arrow assignment to create what I called `throws2` below, or put something like `throws2=` on the beginning of the line. Your choice.

```
throws %>% mutate(diff=baseball-softball) ->
throws2
```

- (c) Carry out a sign test in R, testing the null hypothesis that the median difference is zero, against the alternative that it is greater than zero. Obtain a P-value. Your option whether you use `smmr` or not.

Solution:

I think using `smmr` is way easier, so I'll do that first. There is even a shortcut in that the null median defaults to zero, which is exactly what we want here:

```
library(smmr)
sign_test(throws2,diff)

## $above_below
## below above
##      2      21
##
## $p_values
## alternative      p_value
## 1          lower 9.999971e-01
## 2           upper 3.302097e-05
## 3    two-sided 6.604195e-05
```

We want, this time, the upper-tailed one-sided test, since we want to prove that students can throw a baseball a *longer* distance than a softball. Thus the P-value we want is 0.000033.

To build it yourself, you know the steps by now.

First step is to count how many differences are greater and less than zero:

```
table(throws2$diff>0)

##
## FALSE  TRUE
##      3      21
```

or

```
table(throws2$diff<0)

##
## FALSE  TRUE
##     22      2
```

or, since we have things in a data frame,

```
throws2 %>% count(diff>0)

## # A tibble: 2 x 2
##   `diff > 0`      n
##   <lg1>      <int>
## 1 FALSE          3
## 2 TRUE          21
```

or count those less than zero. I'd take any of those.

Note that these are *not all the same*. One of the differences is in fact exactly zero. The technically right thing to do with the zero difference is to throw it away (leaving 23 differences with 2 negative and 21 positive). I would take that, or 2 or 3 negative differences out of 24 (depending on whether you count “greater than zero” or “less than zero”). We hope that this won’t make a material difference to the P-value; it’ll make some difference, but won’t (we hope) change the conclusion about whether to reject.

Second step is to get a P-value for whichever one of those you got, from the appropriate binomial distribution.

The P-value is the probability of getting 21 (or 22) positive differences out of 24 (or 23) or more, since this is the end of the distribution we should be at if the alternative hypothesis is correct. Thus any of these will get you a defensible P-value:

```
sum(dbinom(21:23,23,0.5))
## [1] 3.302097e-05
sum(dbinom(22:24,24,0.5))
## [1] 1.7941e-05
sum(dbinom(21:24,24,0.5))
## [1] 0.0001385808
sum(dbinom(0:2,23,0.5))
## [1] 3.302097e-05
sum(dbinom(0:2,24,0.5))
## [1] 1.7941e-05
sum(dbinom(0:3,24,0.5))
## [1] 0.0001385808
```

The first and fourth of those are the same as `smmr` (throwing away the exactly-median value).

As we hoped, there is no *material* difference here: there is no doubt with any of these possibilities that we will reject a median difference of zero in favour of a median difference greater than zero.

6. The tropical flower *Heliconia* is fertilized by hummingbirds, a different species for each variety of *Heliconia*. Over time, the lengths of the flowers and the form of the hummingbirds’ beaks have evolved to match each other. The length of the *Heliconia* flower is therefore an important measurement. Does it have a normal distribution for each variety?

The data set at <http://www.utsc.utoronto.ca/~butler/c32/heliconia.csv> contains the lengths (in millimetres) of samples of flowers from each of three varieties of *Heliconia*: *bihai*, *caribaea* red, and *caribaea* yellow.

- (a) Read the data into R. There are different numbers of length measurements for each variety. How does this show up in the data frame? (Look at all the rows, not just the first ten.)

Solution: The usual `read.csv`:

```

heliconia=read_csv("heliconia.csv")

## Parsed with column specification:
## cols(
##   bihai = col_double(),
##   caribaea_red = col_double(),
##   caribaea_yellow = col_double()
## )

```

I suggested to look at *all* the rows. Here's why:

```

heliconia %>% print(n=Inf)

## # A tibble: 23 x 3
##   bihai caribaea_red caribaea_yellow
##   <dbl>         <dbl>         <dbl>
## 1  47.1          41.9          36.8
## 2  46.8          42.0          37.0
## 3  46.8          41.9          36.5
## 4  47.1          43.1          36.1
## 5  46.7          41.5          36.0
## 6  47.4          41.7          35.4
## 7  46.4          39.8          38.1
## 8  46.6          40.6          37.1
## 9  48.1          39.6          35.2
## 10 48.3          42.2          36.8
## 11 48.2          40.7          36.7
## 12 50.3          37.9          35.7
## 13 50.1          39.2          36.0
## 14 46.3          37.4          34.6
## 15 46.9          38.2          34.6
## 16 48.4          38.1           NA
## 17 NA           38.1           NA
## 18 NA           38.0           NA
## 19 NA           38.8           NA
## 20 NA           38.2           NA
## 21 NA           38.9           NA
## 22 NA           37.8           NA
## 23 NA           38.0           NA

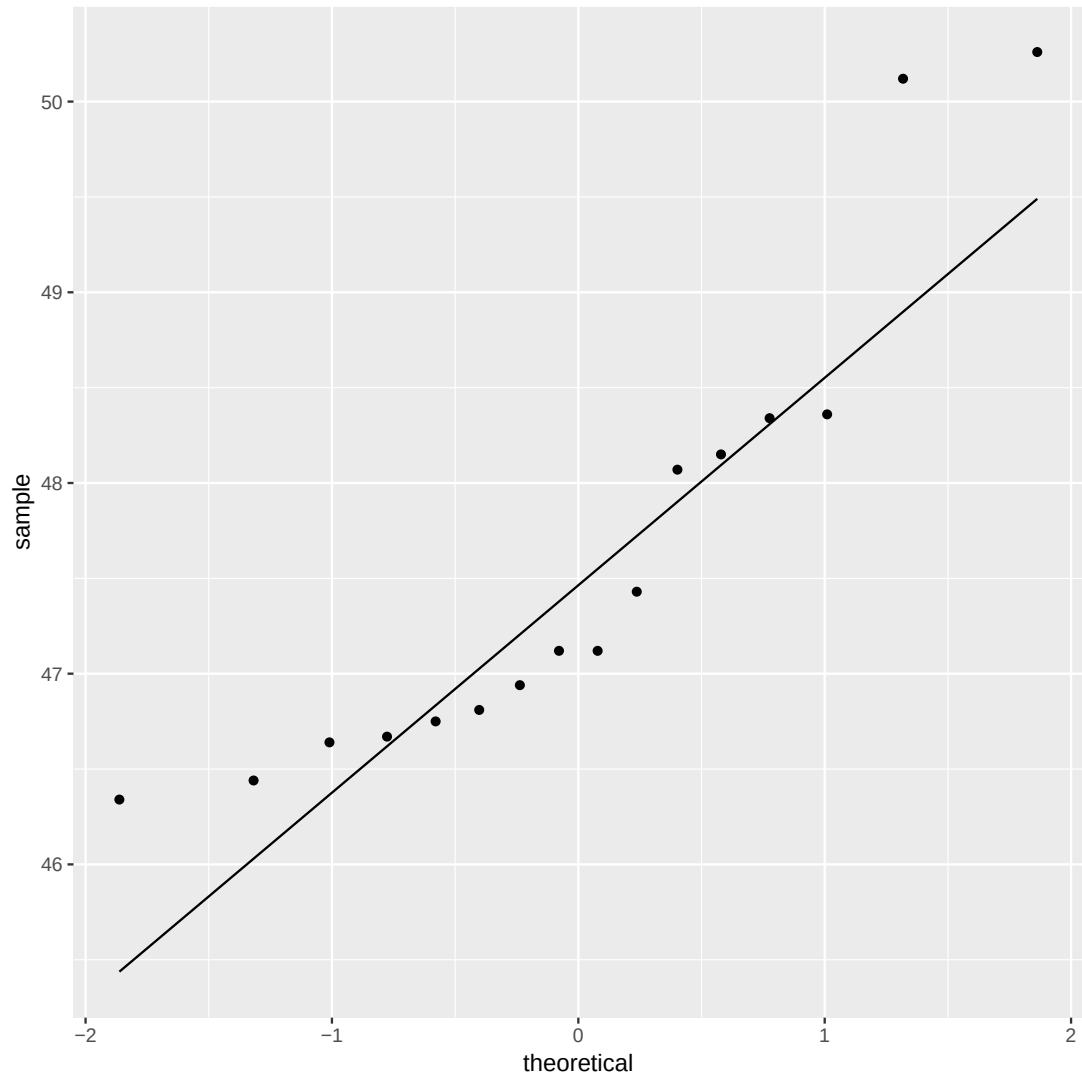
```

The varieties with fewer values have missings (NAs) attached to the end. This is because all the columns in a data frame have to have the same number of values. (The missings won't impact what we do below — we get a warning but not an error, and the plots are the same as they would be without the missings — but you might be aesthetically offended by them, in which case you can read what I do later on.)

- (b) Make a normal quantile plot for the variety *bihai*.

Solution: There's a certain amount of repetitiveness here (that we work around later):

```
ggplot(heliconia, aes(sample=bihai))+stat_qq()+stat_qq_line()
## Warning: Removed 7 rows containing non-finite values (stat_qq).
## Warning: Removed 7 rows containing non-finite values (stat_qq_line).
```

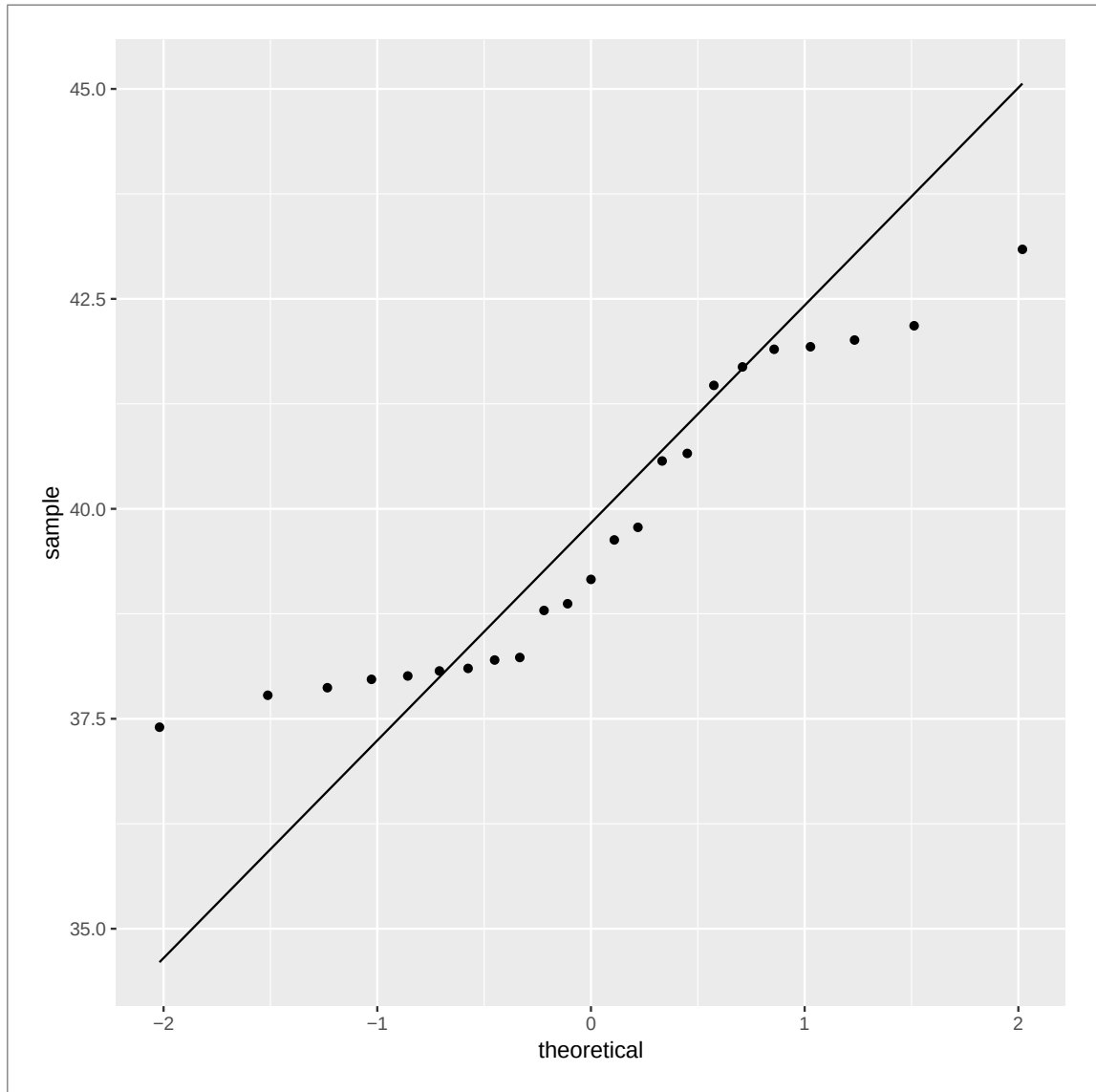


I'm saving the comments until we've seen all three.

- (c) Make a normal quantile plot for the variety *Caribaea* red (note that the variable name in the data frame has an underscore in it).

Solution: Same idea again:

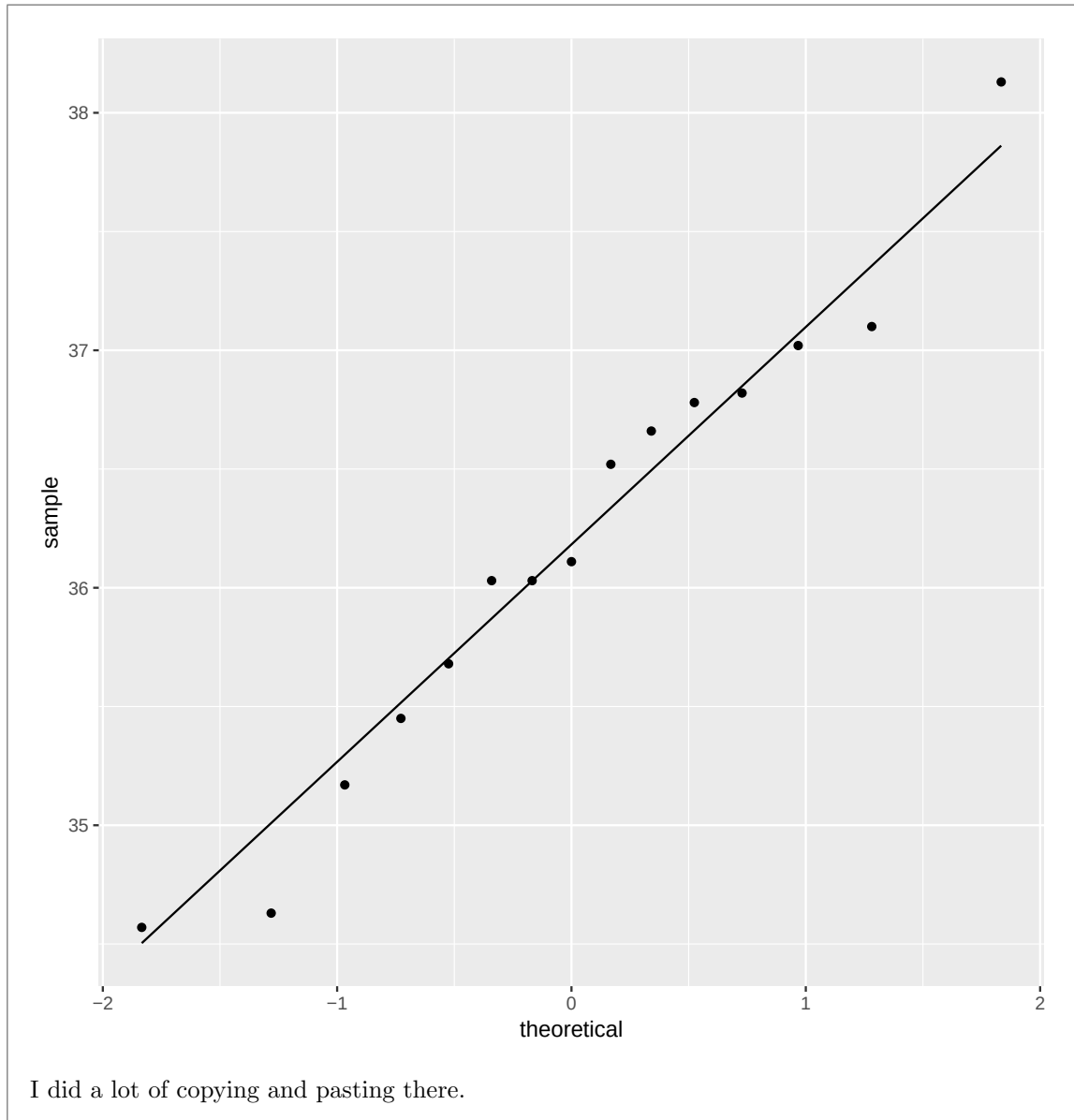
```
ggplot(heliconia, aes(sample=caribaea_red))+stat_qq()+stat_qq_line()
```



(d) Make a normal quantile plot for the variety *Caribaea* yellow (this also has an underscore in it).

Solution: And, one more time:

```
ggplot(heliconia, aes(sample=caribaea_yellow)) + stat_qq() + stat_qq_line()  
## Warning: Removed 8 rows containing non-finite values (stat.qq).  
## Warning: Removed 8 rows containing non-finite values (stat.qq_line).
```



- (e) Which of the three varieties is closest to having a normal distribution? Explain (very) briefly.

Solution: Look at the three plots, and see which one stays closest to the line. To my mind, this is clearly the last one, *Caribaea* yellow. So your answer ought to be “*Caribaea* yellow, because the points are closest to the line”. This, I would say, is acceptably close to normal, so using a *t*-test here would be fine.

The answer “the last one” is not quite complete, because I asked you which *variety*, so your answer needs to name a variety.

- (f) For each of the two other varieties, apart from the one you mentioned in the last part, describe briefly how their distributions fail to be normal.

Solution: Let's look at *bihai* first. I see this one as an almost classic curve: the points are above the line, then below, then above again. If you look at the data scale (*y*-axis), the points are too bunched up to be normal at the bottom, and too spread out at the top: that is, skewed to the *right*.

You might also (reasonably) take the view that the points at the bottom are close to the line (not sure about the very smallest one, though), but the points at the top are farther away, so that what we have here is two outliers at the top. I'm OK with that.

It's often difficult to distinguish between skewness and outliers (at the end of the long tail). What you conclude can often depend on how you look.

We also had to look at the second plot, *caribaea* red. This is a rather strange one: the points veer away from the line at the ends, but look carefully: it is *not* outliers at both ends, but rather the points are *too bunched up* to be normal at both ends: that is, the distribution has *short tails* compared to the normal. It is something more like a uniform distribution, which has no tails at all, than a normal distribution, which won't have outliers but it *does* have some kind of tails. So, "short tails".

Extra: that's all you needed, but I mentioned above that you might have been offended aesthetically by those missing values that were not really missing. Let's see if we can do this aesthetically. As you might expect, it uses several of the tools from the "tidyverse".

First, tidy the data. The three columns of the data frame are all lengths, just lengths of different things, which need to be labelled. This is `gather` from `tidyr`.

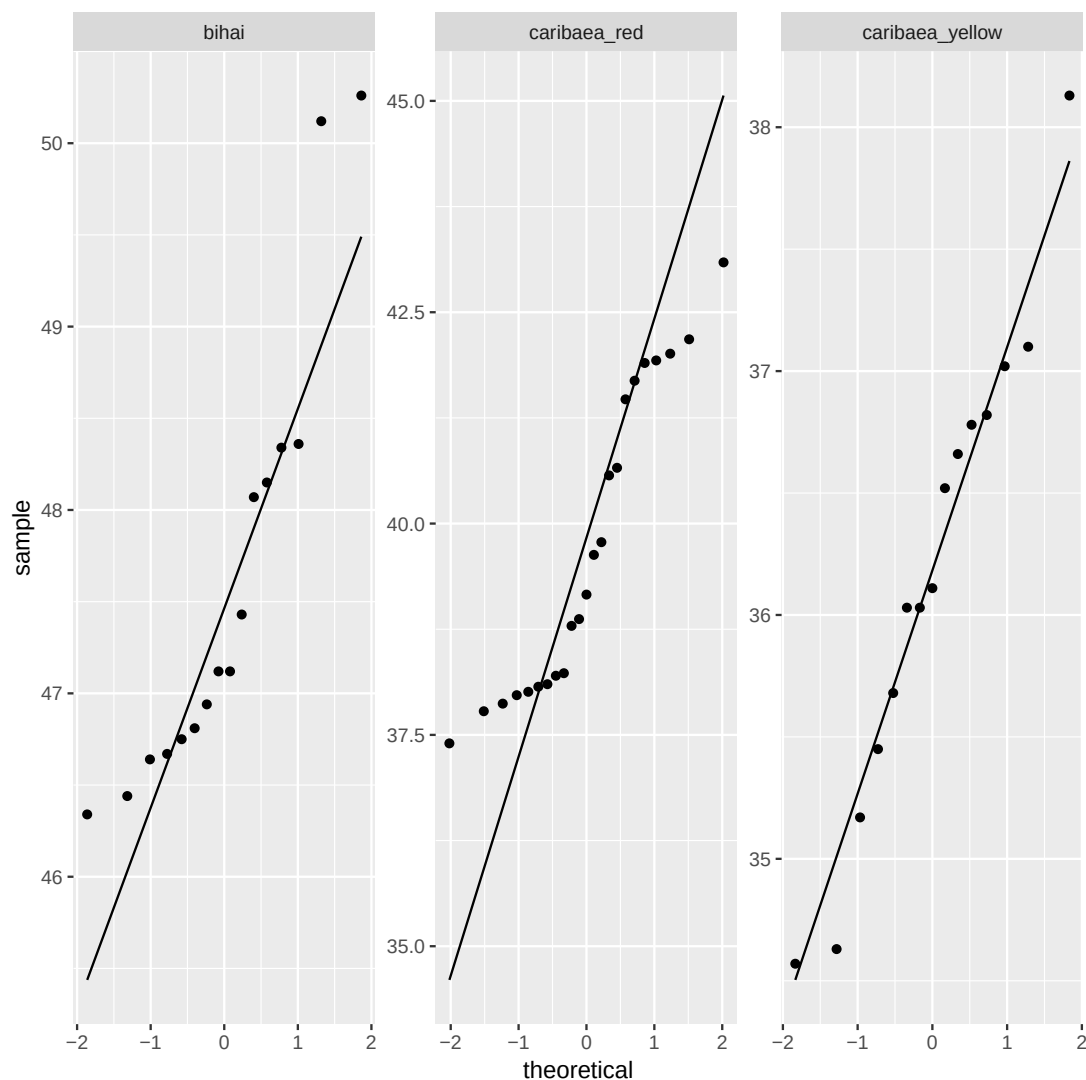
```
heliconia.long=heliconia %>%
  gather(variety,length,bihai:caribaea_yellow,na.rm=T)
heliconia.long

## # A tibble: 54 x 2
##   variety length
##   * <chr>    <dbl>
## 1 bihai      47.1
## 2 bihai      46.8
## 3 bihai      46.8
## 4 bihai      47.1
## 5 bihai      46.7
## 6 bihai      47.4
## 7 bihai      46.4
## 8 bihai      46.6
## 9 bihai      48.1
## 10 bihai     48.3
## # ... with 44 more rows
```

This is now aesthetic as well as tidy: all those NA lines have gone (you can check that there are now $16 + 23 + 15 = 54$ rows of actual data, as there should be).

Now, how to get a normal quantile plot for each variety? This is `facet_wrap` on the end of the `ggplot` again.

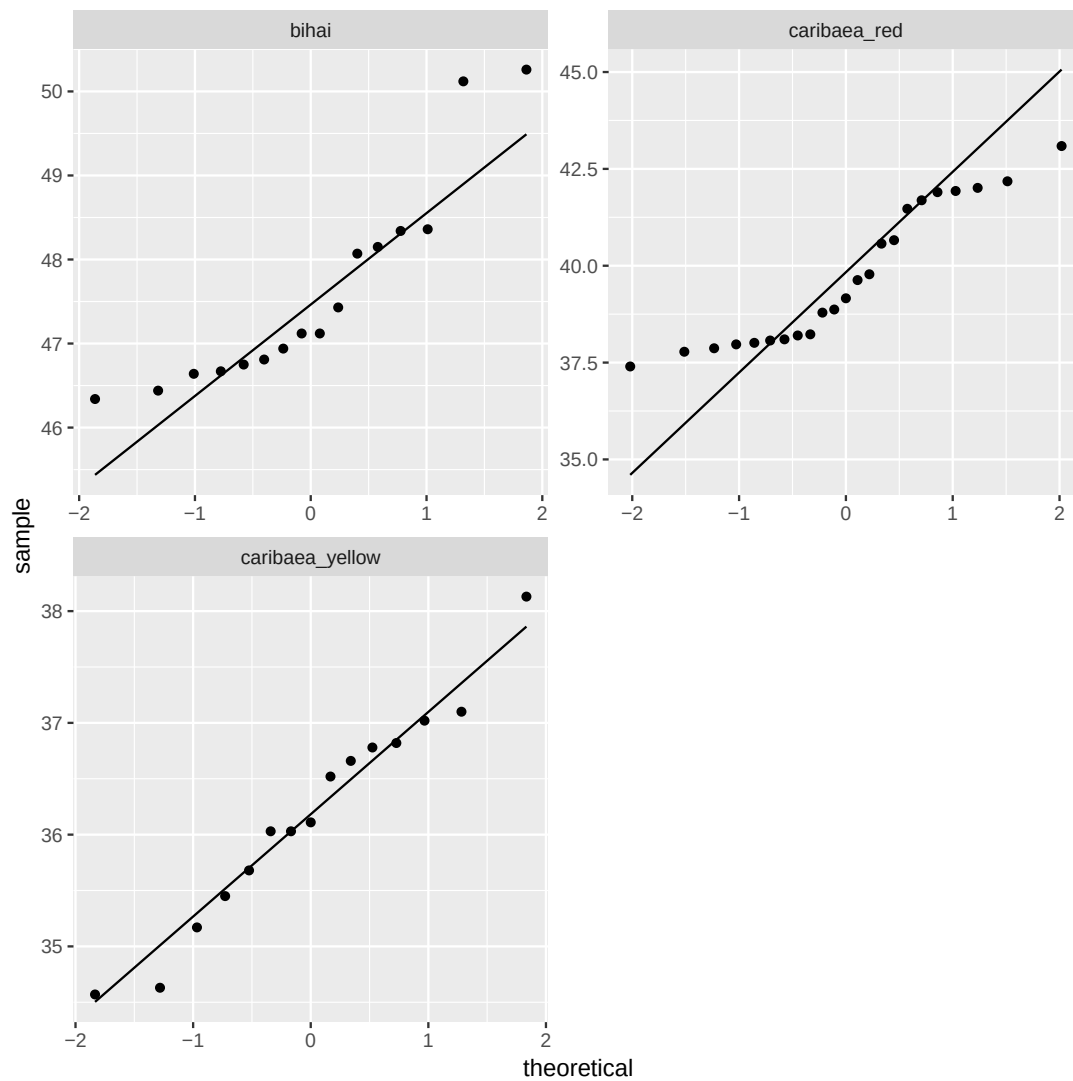
```
ggplot(heliconia.long,aes(sample=length))+
  stat_qq()+stat_qq_line()+
  facet_wrap(~variety,scale="free")
```

These are a bit elongated vertically. The `scale="free"` allows a different vertical scale for each plot (otherwise there would be one vertical scale for all three plots); I decided that was best here since the typical lengths for the three varieties are different. *Caribaea* yellow is more or less straight, *bihai* has outliers (and may also be curved), *caribaea* red has that peculiar S-bend shape.

I didn't really like the vertical elongation. I'd rather have the plots be almost square, which they would be if we put them in three cells of a 2×2 grid. `facet_wrap` has `nrow` and `ncol` which you can use one or both of to make this happen. This creates an array of plots with two columns and as many rows as needed:

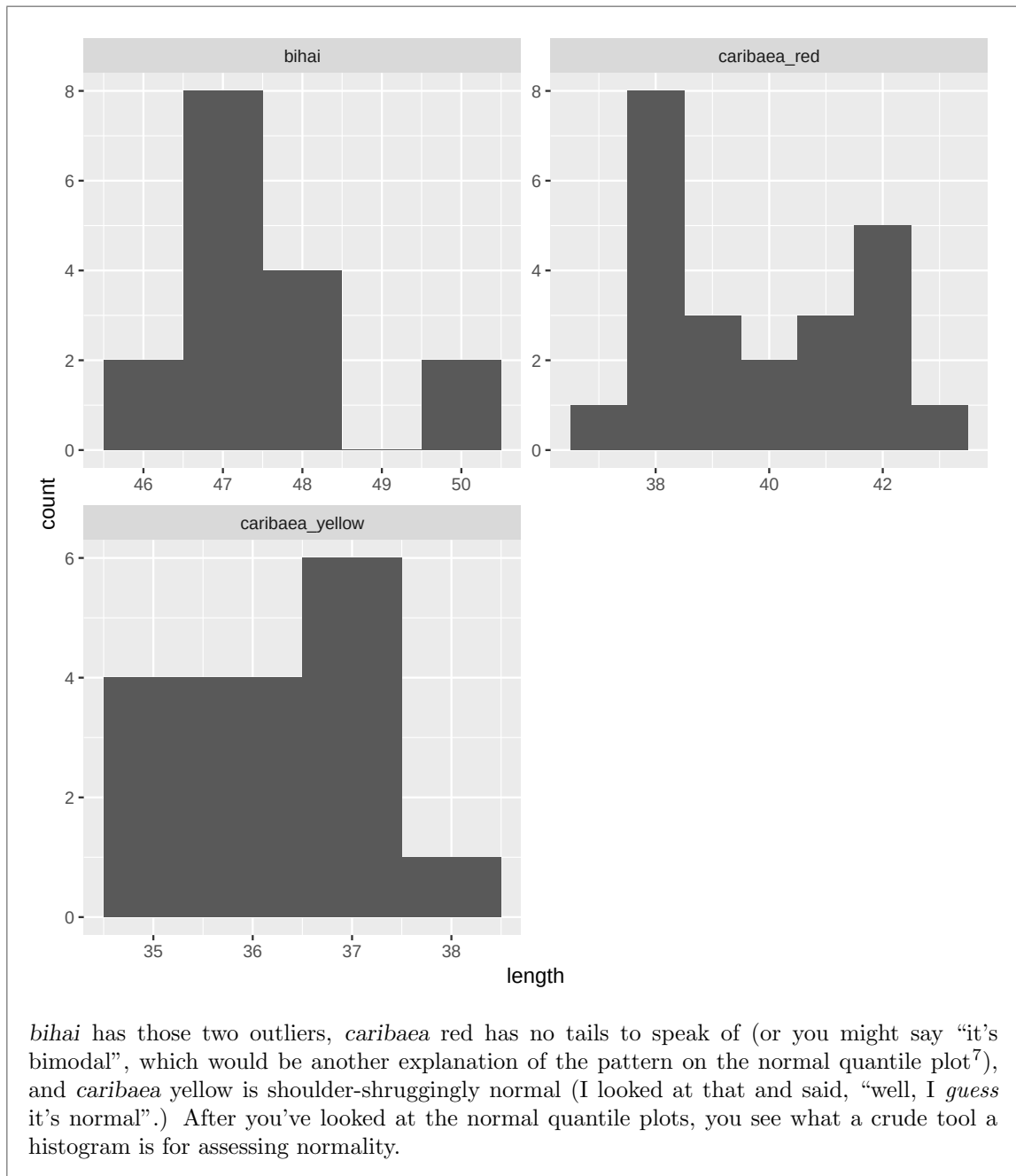
```
ggplot(heliconia.long, aes(sample=length))+
  stat_qq()+stat_qq_line()+
  facet_wrap(~variety, scale="free", ncol=2)
```



I think the square plots make it easier to see the shape of these: curved, S-bend, straightish.

Almost the same code will get a histogram for each variety, which I'll also make squarish:

```
ggplot(heliconia.long, aes(x=length)) +  
  geom_histogram(binwidth=1) + facet_wrap(~variety, scale="free", ncol=2)
```



7. Sunspots have been observed for many centuries; indeed, Persian and Chinese astronomers kept records going back thousands of years. Some archaeologists think sunspot activity may somehow be related to periods of drought in the southwestern US. Data for the average number of sunspots observed in a four-week period, for 40 such periods, are given in <http://www.utsc.utoronto.ca/~butler/c32/sunspots.txt>. These values date from Spanish colonial times.

There is only one column of data.

- (a) Read the data into R, displaying the first ten rows. (The one column of the data frame is called **spots**, so you can safely call the entire data frame **sunspots**.)

Solution: Use `read_csv` or `read_delim` or whatever you like. They both work, since there is only one column of data:

```
this_url="http://www.utoronto.ca/~butler/c32/sunspots.txt"
sunspots=read_csv(this_url)

## Parsed with column specification:
## cols(
##   spots = col_double()
## )

sunspots

## # A tibble: 40 x 1
##   spots
##   <dbl>
## 1  12.5
## 2  14.1
## 3  37.6
## 4  48.3
## 5  67.3
## 6   70
## 7  43.8
## 8  56.5
## 9  59.7
## 10  24
## # ... with 30 more rows
```

That looks good.

Even something crazy like this works, since you can pretend the columns are separated by anything:

```
xx=read_delim(this_url,"?")

## Parsed with column specification:
## cols(
##   spots = col_double()
## )

xx

## # A tibble: 40 x 1
##   spots
##   <dbl>
## 1  12.5
## 2  14.1
## 3  37.6
## 4  48.3
## 5  67.3
## 6   70
## 7  43.8
## 8  56.5
## 9  59.7
## 10  24
## # ... with 30 more rows
```

- (b) Obtain the median and mean number of sunspots.

Solution: Just `summarize` — no need for a `group_by`:

```
sunspots %>% summarize(mean=mean(spots),med=median(spots))  
  
## # A tibble: 1 x 2  
##   mean  med  
##   <dbl> <dbl>  
## 1  47.0  44.6
```

Call the mean and median whatever you like.

The mean is bigger than the median, which is a possible suggestion of right-skewness. Not a very strong one, though, since we don't know how much variability there is.

- (c) The long-term average number of sunspots in a four-week cycle is 41. Carry out a sign test, using `smmr`, to test that the (population) median is 41 for these data, against the alternative hypothesis that it is not 41. What do you conclude, in the context of the data?

Solution: I made the rest of the question easier than it was before (since I thought it was getting too long). This way, you just have to use `smmr` to get all the answers. This test is two-sided:

```
library(smmr)  
sign_test(sunspots,spots,41)  
  
## $above_below  
## below above  
##    19    21  
##  
## $p_values  
##   alternative   p_value  
## 1         lower 0.6820860  
## 2          upper 0.4373147  
## 3    two-sided 0.8746293
```

The P-value should be the one on the two-sided line, 0.8746. There is no evidence that the median number of sunspots in Spanish colonial times is different from the long-term average.

`smmr` also gives you a count of how many data values are above and below the hypothesized median. These are very close to an even split above and below, which is why there is no way to reject a median of 41.

If that doesn't work for you, you can go this way, to get the same answer:

```
sign_test0(sunspots$spots,41)

## $above_below
## below above
##      19      21
##
## $p_values
##   alternative   p_value
## 1         lower 0.6820860
## 2          upper 0.4373147
## 3    two-sided 0.8746293
```

which gets exactly the same answer and is thus equally acceptable.

- (d) Use `smmr` to find a 90% confidence interval for the median.

Solution: This:

```
ci_median(sunspots,spots,conf.level=0.90)
## [1] 25.70298 53.99511
```

That's all I needed. It's a rather long confidence interval, since there is a lot of variability.

Alternatively, do this:

```
ci_median0(sunspots$spots,conf.level=0.90)
## [1] 25.70298 53.99511
```

By way of background, remember how this CI is actually found: it's all the values of the population median that *would not* get rejected at $\alpha = 0.10$.

For example, this means that a null median of 55 would be rejected at $\alpha = 0.10$, and a null median of 53 would not be.

The obvious way to verify this is to re-run `sign_test` twice more, once with a null median of 55 and once with 53:

```

sign_test(sunspots,spots,55)

## $above_below
## below above
##      27      13
##
## $p_values
##   alternative    p_value
## 1         lower 0.01923865
## 2         upper 0.99170550
## 3    two-sided 0.03847731

sign_test(sunspots,spots,53)

## $above_below
## below above
##      24      16
##
## $p_values
##   alternative    p_value
## 1         lower 0.1340936
## 2         upper 0.9230700
## 3    two-sided 0.2681873

```

You also have the option of using a pipe, in which case the initial data frame disappears, for example:

```

sunspots %>% sign_test(spots,53)

## $above_below
## below above
##      24      16
##
## $p_values
##   alternative    p_value
## 1         lower 0.1340936
## 2         upper 0.9230700
## 3    two-sided 0.2681873

```

There is also the short-cut just to get the two-sided P-value, which is equally good, perhaps better even. The hypothesized median goes *first* in this one:

```

pval_sign(55,sunspots,spots)
## [1] 0.03847731

pval_sign(53,sunspots,spots)
## [1] 0.2681873

```

In all of these, the two sided P-value for 55 is 0.038, less than 0.10, so that 55 is outside, and the two-sided P-value for 53 is 0.268, bigger than 0.10, so 53 is *inside*.

Because we're talking about a 90% CI, you need to compare the P-values with *0.10* rather than 0.05. (Also, the test is two-sided already, so there is no halving or doubling involved.)

As I wrote `ci_median`, it actually *uses* `pval_sign`, so the procedure ought to be entirely con-

sistent doing it this way and using `ci_median`. I decided to make `pval_sign` “public”⁸ in case we ever needed to use it ourselves.⁹

The question as I originally wrote it asked you to do this, but on reflection I thought that the question was getting too long, so I simplified it for you. The trial-and-error way of getting the interval itself either involves bisection at each end (which is what `ci_median` does), or you can set up a big for-each, like this:

```
tibble(medians=seq(20,60,5)) %>%
  mutate(pval=map_dbl(medians, ~pval_sign(., sunspots, spots)))

## # A tibble: 9 x 2
##   medians    pval
##   <dbl>    <dbl>
## 1     20 0.00643
## 2     25 0.0807
## 3     30 0.430
## 4     35 0.430
## 5     40 0.875
## 6     45 1.13
## 7     50 0.337
## 8     55 0.0385
## 9     60 0.00643
```

I’m not quite sure about the P-value greater than 1. I think this happened because there was an *exactly even* split of values above and below 45, and the probability of less-or-equal an even split and greater-or-equal are both slightly bigger than 0.5.

Anyway, to this accuracy, the 90% CI is from 30 to 50. If you look at a bunch of null medians between 25 and 30, and between 50 and 55, you can nail down the endpoints more precisely.

I wonder how the (incorrect) *t* CI compares, for the mean?

```
t.test(sunspots$spots, conf.level=0.90)

##
## One Sample t-test
##
## data: sunspots$spots
## t = 7.9287, df = 39, p-value = 1.179e-09
## alternative hypothesis: true mean is not equal to 0
## 90 percent confidence interval:
## 37.04383 57.03617
## sample estimates:
## mean of x
## 47.04
```

The CI for the mean is (i) higher at both ends, and (ii) a bit shorter. But (ii) is metaphorical fool’s gold, since we don’t trust *t* here.

8. How accurately do male students report their heights? Twelve male students at a university were asked to report their heights in inches. After they did this, the experimenter accurately measured their heights, and recorded both the report and the accurate measurement for each student. The data, separated by spaces, are given in <http://www.utsc.utoronto.ca/~butler/c32/heights.txt>.

(a) Read the data into R and display your data frame (or the first ten rows, if that’s all that displays).

Solution: The usual. Data values separated by spaces:

```
myurl="http://www.utoronto.ca/~butler/c32/heights.txt"
heights=read_delim(myurl," ")

## Parsed with column specification:
## cols(
##   reported = col_double(),
##   measured = col_double()
## )

heights

## # A tibble: 12 x 2
##   reported measured
##   <dbl>    <dbl>
## 1      67      66.3
## 2      73      72.8
## 3     67.5      68
## 4      70      69.4
## 5      68      67.3
## 6      70      70.2
## 7      71      70.9
## 8      68      67.6
## 9     67.5      67.1
## 10     66      65.3
## 11     70      68.8
## 12    71.5      70.9
```

- (b) Test whether there is any difference between the mean height reported and the mean height measured. What do you conclude?

Solution: This one is trickier than the two-sample one, because you can't use a "model formula" with a squiggle and a `data=`; you have to go back to `with` or refer to things with dollar signs. I like the `with` way better, since then you have a "context" for your two columns (a data frame that they live in):

```
with(heights,t.test(reported,measured,paired=T))

##
## Paired t-test
##
## data: reported and measured
## t = 3.1007, df = 11, p-value = 0.01009
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.1184863 0.6981804
## sample estimates:
## mean of the differences
##                0.4083333
```

0.01009 is less than the usual 0.05, so we *reject* the null hypothesis and declare that there is a difference between reported and measured heights. (This is correctly a two-sided test.)

You can just as well compare measured and reported the other way around; the differences will all be of opposite signs, but the P-value will be the same.

Extra: to see which way the difference goes, look at the confidence interval. Mine contains positive values; it says that the reported heights are between 0.1 and 0.7 inches taller than the measured ones on average. There are a couple of things to take from this: one, the men concerned are pretty accurate at reporting their heights (less than an inch off on average), but also they tend to overestimate how tall they are by a little. This might be because they measured themselves accurately at some point, but then rounded up.

Another way to do the test is to calculate the differences, and then test whether they have mean *zero*. This is a little fiddly with *tidyverse* ideas, since the one-sample `t.test` doesn't take a data frame first:

```
heights %>% mutate(the_diff=reported-measured) %>%  
  pull(the_diff) %>%  
  t.test(mu=0)  
  
##  
## One Sample t-test  
##  
## data: .  
## t = 3.1007, df = 11, p-value = 0.01009  
## alternative hypothesis: true mean is not equal to 0  
## 95 percent confidence interval:  
## 0.1184863 0.6981804  
## sample estimates:  
## mean of x  
## 0.4083333
```

Same P-value again.

The tricky part was to recognize that a one-sample `t.test` requires a *column* first (and not a data frame: it doesn't take a `data=` at any point, unlike the two-sample `t`). So first we calculate the column of differences and then we pull out just that column (as a vector), and feed that into `t.test` as its “invisible” first input (the “whatever came out of the previous step”). Most of the Tidyverse things do something to a data frame and give you back another data frame, but `pull` returns you a single column in a vector (and `pluck` returns you a single value by column and row).

Hand the next two questions in:

9. In 1999, the National Basketball Association introduced some rule changes that were intended to open up the game and increase scoring. You can read about them here: <https://www.nytimes.com/1999/10/31/sports/1999-2000-nba-preview-playing-by-the-new-rules.html>. I especially like the phrase “borderline thuggery”. Before 1999 the average number of points per game (both teams together) was 183.2. There were 25 games played during the period December 10–12, 1999,¹⁰ which you can take as being a random sample of “all possible games” played under the new rules. The data are in <http://www.utsc.utoronto.ca/~butler/c32/nbapoints.csv>.
 - (a) (2 marks) Read in and display (some of) the data. What do you have?
 - (b) (2 marks) Make a normal quantile plot of the points scored in each game.
 - (c) (2 marks) What do you notice about the distribution of the number of points scored? What does that tell you about an appropriate test to do? Explain briefly.

- (d) (2 marks) Do the appropriate test, defending your choice of a one-sided or two-sided test. Use `smmr` if appropriate. What do you conclude, in the context of the data?
10. A company is growing and would like to attract more employees. The company would like to advertise that salaries there are increasing. To do this, the company randomly samples 20 employees that have been working there since January 2016, and for each of these employees, records their salary in January 2016 and January 2017. The data, with salaries in thousands of dollars, are in <https://www.utsa.edu/~butler/c32/salaryinc.txt>.
- (a) (2 marks) Read the data into R and demonstrate that you have two salaries for each of 20 employees.
- (b) (2 marks) To compare the salaries, explain briefly why a matched-pairs test would be better than a two-sample test.
- (c) (4 marks) Make a suitable graph to assess the assumptions for a matched-pairs t -test. What does your graph tell you?
- (d) (3 marks) Carry out a suitable matched-pairs t -test on these data. (If you thought in the previous part that this was the wrong thing to do, do it anyway for the purposes of this assignment.) What do you conclude?
- (e) (2 marks) The company would like to estimate *how much* salaries are increasing, on average. Obtain some output that will enable the company to assess this, and tell the CEO which piece of the output they should look at.

Notes

¹You can change this by adding something like `tol=1e-4` to the end of your `ci_median`.

²Meaning, I forget how to do it. But it has something to do with looking at alternatives that are very close to the null.

³If you've ever run into the exponential distribution, you'll recall that this is right skewed with a very long tail. The Laplace distribution looks like two of these glued back to back.

⁴This is about the *only* way in which the normal and Laplace distributions are alike.

⁵My hat stays on my head.

⁶I learned yesterday that the Welsh word for "ironing" is *smwddio*, which seems weird until you say it out loud: it sounds like "smoothio".

⁷If you have studied a thing called "kurtosis", the fourth moment about the mean, you'll know that this measures *both* tail length *and* peakedness, so a short-tailed distribution also has a strong peak. Or, maybe, in this case, two strong peaks.

⁸The R term is "exporting" functions for users outside a package. `smmr` has some functions that I didn't export, since only I would need to use them, but it has some "extras" that I did export, such as this one.

⁹`pval_sign` has an odd order of inputs; instead of having the data frame, then the column, then the null median, it has the null median first. This is because, when I wrote it in summer 2017, I hadn't mastered the "squiggle" version of `map_dbl`, and if you use that with a named function, the thing that is "for each" has to go first. I knew I would be making the CI by trying a bunch of null medians, hence I wrote the function as I did. And we're stuck with it now!

¹⁰After the rule changes were implemented, and far enough into the season to allow the players and officials to get used to them.

¹¹The point totals are all integers, so the population median ought to be an integer or integer-plus-half. 183.2 is probably a mean. But we ignore that here.

¹²Major League Baseball does this kind of thing all the time, since they like to maintain what they consider an appropriate balance between batting and pitching.